



Based on all the issues identified, assess whether the model is ready for production. List the mandatory fixes required before deployment.

Kumar Yash	1RV23AI047
M P Sahasra	1RV23AI060
Neelam J	1RV23AI066
Pranav Pratyush	1RV23AI133



Case Study – MedInsure Health Claim Approval Model

MedInsure, a health-insurance company, has developed a machine-learning model to automatically score and approve low-risk claims under ₹ 25,000. The model was trained using five years of historical claim records, hospital types, diagnosis codes, treatment cost ranges, customer risk profiles, and fraud-flag indicators. During development, the model achieved strong accuracy and passed internal validation. Now, the company plans to move the model to the production environment.



1. Runtime Environment Mismatch: Predictions from the staging server differ significantly from the development notebook. The staging environment uses Python 3.10, while the model was trained using Python 3.8, and several ML library versions differ.

Issue Overview

- Staging environment uses **Python 3.10**; model was trained on **Python 3.8**.
- **Version drift** in core ML libraries (e.g., NumPy, Pandas, scikit-learn).
- Results in **non-deterministic inference outputs** and **validation irreproducibility**.

Technical Impact

- **API/ABI incompatibility** causing numerical instability and altered algorithmic behaviour.
- Breaks **environment parity**, invalidating all prior QA metrics.
- Violates **reproducibility**, **auditability**, and **model lineage** requirements.

Root Cause

- Lack of **immutable runtime** during training and staging.
- Dependencies not frozen via **hash-locked manifests**.
- No **containerized training-to-serving artifact**.

Mandatory Fixes

- Enforce **containerized execution** (Docker) with pinned Python + dependency versions.
- Use **dependency lockfiles** (Poetry, pip-tools) to eliminate version drift.
- Register model in a **Model Registry** (MLflow/SageMaker) to store environment spec, artifacts, and metadata.
- Run validation **inside the same container image** used for production deployment.



2. Data Access & Schema Changes: In staging, the “hospital_risk_score” feature is missing because the ETL pipeline was updated last month, but the model still expects this feature. As a result, the model rejects 22% of claim requests.

Issue Overview

- Upstream ETL removed the **hospital_risk_score** feature.
- Model’s feature schema no longer aligns with the production data schema.
- Results in **feature unavailability**, causing **22% inference failures**.

Technical Impact

- **Schema drift** between offline training data and online inference pipeline.
- Violates **feature consistency guarantees** required for stable model serving.
- Causes **pipeline malfunction** due to missing mandatory input vector components.
- Breaks **data lineage**, **data contract**, and **input integrity** checks.

Root Cause

- No enforced **Data Contracts** between Data Engineering and ML pipelines.
- Lack of a **Feature Store** to abstract feature retrieval and manage versioning.
- Staging environment missing required **data access configuration** and connectivity.

Mandatory Fixes

- Implement a **Feature Store** (offline + online) to centralize feature definitions and ensure consistent availability.
- Enforce **Data Contracts / Schema Validation** using tools like Great Expectations or JSON Schema.
- Integrate **schema checks** into CI/CD to block ETL updates that break model dependencies.
- Add **robust fallback logic** (default-value imputation or models tolerant to missing features).



3. Model Risk Evaluation Concerns: The auditor notices the model performs poorly for claims involving “rare surgical procedures” due to insufficient representation in the training data.

Issue Overview

- Model exhibits **low predictive performance** on claims involving **rare surgical procedures**.
- Training dataset lacks sufficient representation → **data sparsity** and **distributional imbalance**.

Technical Impact

- Breach of **Domain of Applicability (DoA)**: inference inputs fall outside the model’s learned distribution.
- Increased **model uncertainty**, **high variance predictions**, and **systemic bias** toward common-case patterns.
- Violates **subpopulation reliability** and **slice-level performance guarantees**.
- Risk of **high-severity misclassification** in edge-case scenarios.

Root Cause

- Lack of **subpopulation-aware evaluation** during validation.
- No **uncertainty quantification** to detect out-of-distribution (OOD) claims.
- Absence of **guardrails** (rule-based constraints or anomaly gating) for rare procedure codes.

Mandatory Fixes

- Deploy an **Anomaly Detection Gatekeeper** (Isolation Forest / One-Class SVM) to detect OOD rare procedures.
- Implement **Conformal Prediction** to produce calibrated confidence intervals.
- Perform **slice-based evaluation** on rare procedures; enforce minimum performance thresholds.
- Introduce **rule-based routing** (manual review) for critical or underrepresented surgical codes.



4. Quality Assurance & Stress Testing: Under high load (5,000 claims/min), the API slows down, causing timeouts for the customer-facing mobile app.

Issue Overview

- API experiences severe latency degradation under **5,000 claims/min** load.
- Leads to **timeout failures** in the customer-facing mobile app.

Technical Impact

- High **inference latency** due to Python-based execution and unoptimized model runtime.
- Failure to meet **throughput, SLA**, and **P95/P99 latency** requirements.
- Indicates lack of **stress testing, load benchmarking**, and **performance assertions** in QA pipeline.
- Violates model-serving **scalability** and **real-time availability** standards.

Root Cause

- Model deployed directly in Python without conversion to an optimized runtime (ONNX/TensorRT).
- No **model compression** (quantization/pruning/distillation) to reduce computational cost.
- Infrastructure missing **autoscaling policies** and high-load resilience.
- QA focused on **average latency**, ignoring **tail latency degradation**.

Mandatory Fixes

- Convert model to **ONNX Runtime** or TensorRT for accelerated inference.
- Apply **quantization, pruning**, and **distillation** to minimize compute overhead.
- Add **load testing** (Locust/JMeter) with strict P95/P99 latency thresholds.
- Configure **autoscaling** (Kubernetes HPA) and implement concurrency-aware serving.
- Optionally deploy a **cascade architecture** where a lightweight model handles common cases.



5. Reproducibility Issues: The auditor requests proof of training data versions and hyperparameters. The data science team cannot fully reproduce the exact training run because environment logs were not captured.

Issue Overview

- Team cannot reproduce the original training run:
 - Missing **training data version**,
 - Missing **hyperparameters**,
 - Missing **environment specification/logs**.
- Auditor request cannot be fulfilled.

Technical Impact

- Violation of **reproducibility**, **model lineage**, and **regulatory auditability** requirements.
- No deterministic pipeline → inability to **debug**, **validate**, or **compare** future model iterations.
- Model artifact is **non-verifiable**, compromising compliance in regulated environments.

Root Cause

- Training executed in **ad-hoc notebook environments** with no automated logging.
- Absent **experiment tracking system** (hyperparameters, metrics, Git commit, environment).
- No **data version control**; training data not snapshotted or hash-addressed.
- Training not managed by **CI/CD Pipelines**, causing inconsistent metadata capture.

Mandatory Fixes

- Integrate **Experiment Tracking** (MLflow / Weights & Biases) to auto-log: hyperparameters, metrics, Git hash, environment specs.
- Implement **Data Version Control** (DVC) or versioned Feature Store to track datasets by hash.
- Run training exclusively through **automated CI/CD pipelines** to guarantee metadata capture.
- Add a **Reproducibility Test**: re-train from stored metadata; reject model if performance deviates beyond tolerance.



6. Security & Adversarial Inputs: Fraud analysts discovered that attackers manipulate diagnosis codes inside claims to trick the model into classifying high-risk claims as low-risk.

Issue Overview

- Attackers manipulate **diagnosis codes** to force misclassification of **high-risk claims as low-risk**.
- Classic **evasion-based adversarial attack** exploiting model decision boundaries.

Technical Impact

- High susceptibility to **adversarial perturbations** in categorical input features.
- Compromises **risk scoring integrity**, enabling large-scale fraudulent exploitation.
- Violates **robustness, input integrity**, and **security-by-design** principles.
- Complex ML model increases **attack surface** and reduces interpretability.

Root Cause

- Absence of **input sanitization** and strict schema enforcement for ICD-10 codes.
- No **adversarial training** or OOD detection in the inference pipeline.
- Unlimited or insufficiently monitored **API query volume**, enabling probing.
- Lack of **fallback mechanisms** for low-confidence or anomalous predictions.

Mandatory Fixes

- Enforce strict **input validation**: allowlisted ICD-10 codes, encoding normalization, feature squeezing.
- Apply **adversarial training** to harden decision boundaries against perturbed inputs.
- Implement **rate limiting**, anomaly detection, and real-time monitoring on the prediction API.
- Use **Champion/Challenger** or hybrid pipelines: parallel simple model checks, confidence thresholds, and manual routing.